Token-aware Agentic AI Systems: A Framework for Evaluating Token Usage Across Orchestration Approaches

No Author Given

No Institute Given

Abstract. Agentic AI systems increasingly automate enterprise workflows by leveraging large language models (LLMs) to orchestrate external tools through standardized interfaces such as the Model Context Protocol (MCP). In this domain, there are two orchestration modalities: direct invocation, where an agent calls tools sequentially and routes each intermediate result through the LLM context, and code-execution invocation, where the agent generates a program that batches multiple tool calls within an isolated sandbox. Although direct invocation is widely adopted and code-execution invocation is an emerging approach, their efficiency trade-offs remain insufficiently studied. This work introduces an evaluation framework for agentic AI systems that compares these orchestration modalities in both single-agent and multi-agent settings, using a heterogeneous task suite (including single-tool, pipeline, and discovery tasks) across multiple MCP servers providing more than 70 tools. The framework includes an LLM-as-a-judge grader and measures token consumption, task quality, and normalized efficiency. With a 0.05 significance level, empirical results indicate that code-execution invocation reduces token consumption while preserving output quality and achieving higher efficiency on complex tasks, particularly pipeline tasks, compared with direct invocation.

Keywords: Agentic AI systems · Evaluation framework · MCP via code execution · Multi-agent systems.

1 Introduction

Nowadays, artificial intelligence (AI) supports and automates enterprise workflows through agentic AI systems [10], which consist of multiple agents that collaborate to carry out interconnected tasks and achieve predefined goals [21]. Agents (endowed with decision-making abilities) establish a context and act toward these goals [7]. In recent years, to enhance their decision-making, agents leverage the reasoning capabilities of large language models (LLMs) [18]. An LLM is a deep learning model trained on a great amount of text to perform natural language tasks [13]. Unlike reactive agents, an LLM-supported agent perceives and establishes context through textual inputs and executes actions accordingly. Frequently, LLM-supported agents generate and monitor plans by interleaving

reasoning traces and actions, following the ReAct framework (see [23] for details). Currently, agentic frameworks such as Amazon Bedrock, AutoGen, LangChain, and Strands Agents allow agents to invoke and orchestrate a myriad of tools and services (ranging from storage systems to AI models and web-scraping utilities) using an open model context protocol (MCP), which has been adopted as an industry standard [12]. MCP enables agents to discover and use tools in a structured manner, decoupling implementation technologies from their invocation.

With MCP providing this layer of interoperability, there are several ways to orchestrate and implement agentic systems (see [2]). In terms of tool invocation, this article focuses on two orchestration modalities: (i) *direct invocation*, in which an agent calls MCP tools one at a time, with each tool's output added back into its context before the next call; and (ii) *code-execution* invocation, in which the agent instead generates a program that invokes multiple tools inside a sandbox, processes intermediate results internally, and returns only the final output to its context.

LLM-supported agents have become increasingly effective at executing enterprise workflows under either orchestration modality. However, their performance depends in part on the amount of information (i.e., the size of the context window, measured in tokens) that an LLM can process in a single request. This constraint has motivated substantial research efforts within the scientific community (see [4], [22], [20]). In this regard, in agentic AI systems executing enterprise workflows, this limitation is further amplified because, in addition to the user's prompt, the context required for direct MCP calls must include detailed tool specifications (e.g., names, descriptions, parameters, and schemas). In practice, these tool descriptions may consume thousands of tokens (occupying a significant portion of the available context window) before an agent can even fulfill a single user request [5]. Moreover, in the context of direct MCP calls, when an LLM-supported agent executes enterprise workflows involving multiple tools, all intermediate results must be routed through the context, i.e., the output of each tool becomes the input of the next, which can lead to context growth that may even exceed the available window size.

To mitigate the above problem, the main objective of this work is twofold: first, to design an evaluation framework for agentic AI systems that compares code-execution invocation and direct invocation as orchestration modalities; and second, to examine whether code-execution invocation offers measurable advantages (both in token consumption and output quality) over direct invocation in agentic systems. To evaluate this, each orchestration modality was tested under both a single-agent and a multi-agent system architecture. Furthermore, to assess the implemented agentic AI systems, the agents were presented with three task types: (i) *single-tool tasks*, involving a single MCP server; (ii) *pipeline tasks*, involving multiple predefined MCP servers; and (iii) *discovery tasks*, also involving multiple MCP servers but requiring the agent to determine which servers (and tools) to invoke. To conduct the experiments, multiple enterprise workflows (involving eight MCP servers and more than 70 tools) were created. The application

domains of these servers and workflow tasks were highly heterogeneous, ranging from Morse code translation to developer utilities.

The contributions of this work are as follows.

- An evaluation framework for agentic AI systems across different orchestration modalities in both single-agent and multi-agent settings.
- Empirical evidence of the trade-offs between two relevant orchestration modalities in agentic systems, namely direct invocation and code-execution invocation.
- Identification of preferable orchestration modalities for different task categories, considering both token consumption and output quality.
- A reproducible evaluation methodology for comparing orchestration modalities in agentic AI systems.

The structure of this article is as follows. Section 2 introduces background concepts and related work; Section 3 describes the evaluation framework; Section 4 presents experimental settings and the empirical results; and Section 5 provides concluding remarks.

2 Background and Related Work

The rise of LLMs has accelerated research on agentic AI systems (see [19], [9], and [1]), which automatically orchestrate web services (commonly referred to as *tools* in this domain) to execute complex workflows. A key paradigm in this context is ReAct [23], where agents alternate between reasoning and tool invocation. This integration of chain-of-thought with tool use has been shown to reduce hallucinations and improve performance on benchmarks such as HotpotQA and ALFWorld. In practice, ReAct-style tool use is commonly implemented through MCP [16], an open protocol providing LLM-supported agents with a uniform interface for discovering and invoking tools via JSON-RPC 2.0. MCP is now supported by major frameworks from OpenAI, Google DeepMind, and Microsoft [12]. However, before executing a workflow, an agent must load tool definitions into its context, which can consume tens of thousands of tokens [6]. As the number of tools grows, so does the token cost of loading specifications and routing intermediate results. Since LLMs are effective at generating code [14], code-execution invocation provides an alternative that reduces context growth by handling intermediate computations programmatically.

In this context, several research efforts have been conducted to evaluate and benchmark agentic AI systems enabled by MCP. Dobriy et al. [8] evaluated LLM-supported agents in the context of federated SPARQL querying. Their system architecture consisted of a single client communicating with a single MCP SPARQL server, primarily evaluating the validity of the generated SPARQL queries. In the domain of mobile computing, Kong et al. [15] proposed MobileWorld, a benchmark for mobile applications using agent-user interaction and MCP servers. With respect to the agentic AI system, their benchmark includes 40 tasks involving 64 tools provided by five MCP servers, evaluating task success

rate and the average number of MCP calls. Mo et al. [17] designed their benchmark to include 95 everyday tasks (e.g., lifestyle, shopping, and leisure activities), in which LLM-supported agents must interact with 70 MCP servers providing 527 tools. As before, their primary performance metric was task success rate, however, they also measured token consumption across several state-of-the-art LLMs, with a particular focus on the costs associated with tool descriptions. Following a similar approach, Guo et al. [11] propose evaluating agentic AI systems using 600 real-world everyday tasks (generated by LLMs but with human supervision). As in [17], the number of MCP servers and tools involved in the benchmark is also considerably large. It is worth mentioning that Guo et al. also propose evaluating token consumption (in addition to task success rate), however, token consumption is assessed from the perspective of the LLMs used without taking into account orchestration modalities.

Whereas the research efforts presented in [8], [15], [17], and [11] are extremely valuable and contribute significantly to the evaluation of agentic AI systems, to the best of the authors' knowledge, no prior work has systematically benchmarked orchestration modalities (namely, direct invocation and code-execution invocation) independently of the underlying AI model, with respect to both token consumption and task/output grade, which is the main focus of the present work.

3 Evaluation Framework

The evaluation framework (Fig. 1) consists of five main components: (i) an LLM-supported agent; (ii) the orchestration modalities: direct invocation and code-execution invocation; (iii) a task suite; (iv) an LLM-as-a-judge grader; and (v) two agent-based system architectures, namely the single-agent and multi-agent setups.

3.1 LLM-supported agent

To execute automated workflows, the LLM-supported agent uses an AI model that receives a prompt describing a task, whose requirements must be aligned with the capabilities of the available MCP servers. The LLM-supported agent also includes a metrics collector to track token usage, success rate, and other relevant performance metrics.

3.2 Orchestration modalities

The LLM-supported agent can make use of two orchestration modalities:

- *Direct invocation* (DI), in which the agent invokes tools individually and sequentially using direct MCP calls, and each call's output (i.e., the emitted tokens) is appended to the agent's context before executing the next call. Using FastMCP, the agent calls the tools (exposed as JSON endpoints) via streamable HTTP or standard I/O calls.

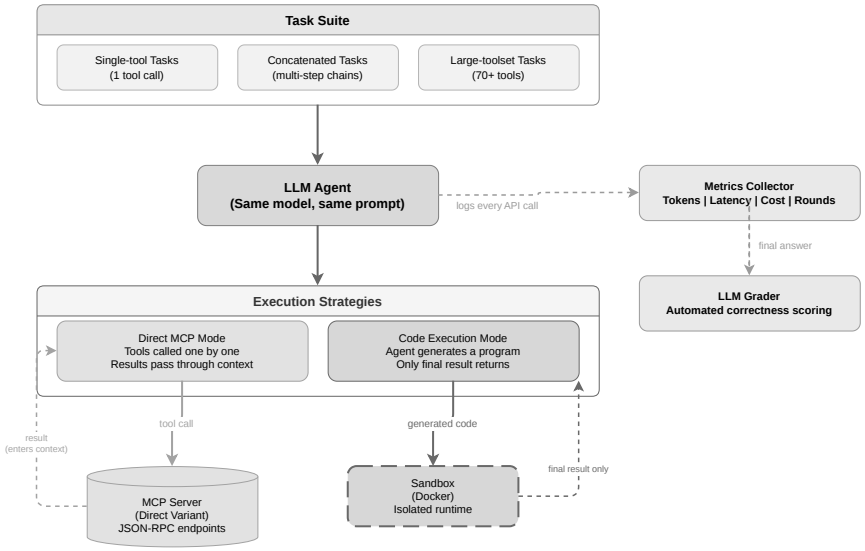

Fig. 1. Architectural components of the proposed evaluation framework.

– *Code-execution invocation* (CI), in which the agent generates a program (treating MCP tools as code APIs that can be invoked programmatically) to orchestrate multiple tool calls within a (Docker) sandbox where the agent executes the code. This allows intermediate results to be processed programmatically within the generated code, and only the final program output, i.e., the resulting tokens, is returned to the agent's context. In addition, inside the Docker container, Python files describe the corresponding MCP tools and a dependency list ensures the agent can import and execute these tools within the sandbox.

3.3 Task suite

The task suite executed by the LLM-supported agent comprises 54 tasks across three task types, each designed to reflect increasing levels of operational complexity:

– *Single-tool*. Tasks where an MCP server exposes a single tool for a straightforward input-output operation. These tasks provide a baseline for comparing the overhead of code-execution invocation against the minimal overhead of a direct MCP call. Examples of single-tool tasks include translating a string to Morse code and searching for an artist on LastFM.
– *Pipeline*. Tasks where an MCP server exposes multiple tools and the agent must chain their outputs sequentially to complete the task. In these tasks, the agent must determine the correct sequence of tool calls and combine them appropriately. An example of a pipeline task is listing an artist's albums on LastFM and then getting the most popular track for each album.

– *Discovery.* Tasks involving 50 or more tools within a single MCP server, requiring the agent to explore the available tools and select the appropriate ones. These tasks may involve either single-tool or pipeline tasks. An example of a discovery task is converting a string using the correct utility from an MCP server containing 60 or more tools.

Nine MCP servers are available to execute the tasks. Each server corresponds to a specific application domain and exposes a different number of tools. Table 1 summarizes the servers used in the evaluation framework.

Table 1. MCP server domains and tool counts.

MCP Server	**Number of tools**	**Description**
Morse	1	Simple morse code translation
Cocktails	3	Cocktail recipe search
Postgres	3	Database operations
NASA	4	NASA image/video search and metadata
AlphaVantage	4	Stock market data
Nutrition	5	Nutritional information lookup
LastFM	26	Music discovery
DevToolkit	60+	Developer utilities
Cross-Server	Varies	Tasks requiring multiple servers

The servers were selected to represent different levels of workflow complexity. The Morse MCP server is the simplest case, exposing a single tool with a straightforward input-output mapping. The DevToolkit MCP server represents the large-toolset scenario, offering more than 60 tools. Postgres and Cross-Server, in contrast, support tasks requiring multi-step reasoning and coordination across multiple MCP tools or servers.

In addition, to evaluate whether the number of connected MCP servers affects performance, each task was executed under three server-load conditions: (i) *minimal*, where only the server(s) required for the task are connected; (ii) *moderate*, where the required server(s) plus a few distractor servers with unrelated tools are connected; and (iii) *maximal*, where the required server(s) plus all remaining servers act as distractors. This setup allows assessing whether loading additional tool definitions into the context degrades performance (one of the key motivations for *code-execution* invocation).

3.4 LLM-as-a-judge grader

The grader receives a rubric specifying how each task should be evaluated and assesses whether the LLM-supported agent produced the correct result with the appropriate level of detail. The rubric includes the task objective and the expected output. Once the LLM-supported agent completes a task, its output is submitted to the LLM-as-a-judge grader, which assigns a score from 0 (lowest) to 1 (highest).

3.5 Agent-based system architectures

The evaluation framework supports two agent-based system architectures: single-agent architecture and multi-agent architecture. Both system architectures can be implemented in Amazon Bedrock [3].

- The *single-agent* architecture is implemented using a *Strands* agent supported by an LLM. The agent receives a prompt that connects the AI model to the available MCP tools required for completing the task. In this setup, the agent has access to all necessary MCP servers. Under *direct invocation*, the Strands agent calls each tool individually via JSON-RPC, appending every result to its context. Under *code-execution invocation*, the agent generates a program that runs inside a Docker sandbox, where the MCP tools are available as importable Python modules. Only the final program output is returned to the agent's context.
- The *multi-agent* architecture is implemented using a *Strands* agent orchestrator. The orchestrator receives the task and delegates subtasks to specialized subordinate agents, each assigned to a specific MCP server. Under *direct invocation*, each subordinate agent calls its corresponding MCP server via JSON-RPC and returns its results to the orchestrator. Under *code-execution invocation*, each subordinate agent operates in its own isolated Docker sandbox containing the necessary tool files and dependencies, returning only the final output. No state is shared between sandboxes.

4 Evaluation and Empirical Results

4.1 Experimental settings

Agent-based system configurations. To determine whether *code-execution invocation* offers measurable advantages over *direct invocation* in agentic AI systems, the task suite described in Section 3.3 was executed using four agent configurations (Fig. 2):

- *Single-agent DI*: Single-agent architecture using direct invocation.
- *Single-agent CI*: Single-agent architecture using code-execution invocation.
- *Multi-agent DI*: Multi-agent architecture using direct invocation.
- *Multi-agent CI*: Multi-agent architecture using code-execution invocation.

Supporting LLM. All four configurations use Claude 3.7 Sonnet through AWS Bedrock with a temperature of 0.2 and the same system prompt.

Performance measures. Each task execution was evaluated using three metrics:

- *Task grade*: A score from 0 to 1 assigned by the LLM-as-a-judge grader.
- *Token usage*: Total input and output tokens consumed across all LLM calls for a given task.
- *Efficiency*: Grade per 1,000 tokens, capturing the trade-off between output quality and token cost.

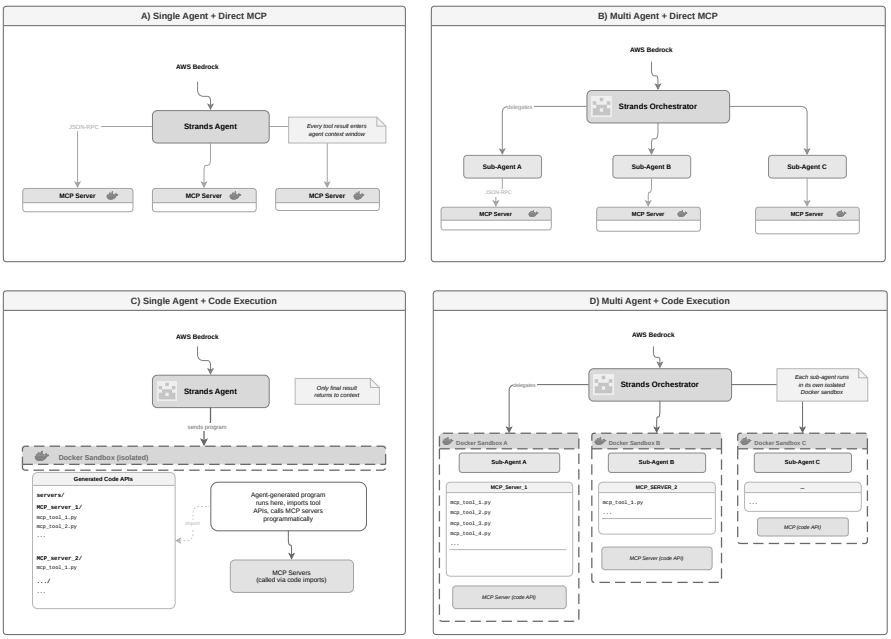

Fig. 2. Overview of the agentic AI system configurations.

Experiment runs. The full experiment consisted of 582 task executions across the four system configurations (Fig. 2), eight MCP servers (Table 1), three task types (single-tool, pipeline, discovery), and three server-load conditions (minimal, moderate, maximal). It should be noted that results from the AlphaVantage MCP server were excluded due to persistent rate-limiting failures.

Empirical results. From the series of experiments, three main observations were obtained. To facilitate interpretation, the results (presented in Figs. 3-5 and Tables 2 and 3) were grouped by task type and orchestration modality. A p-value less than 0.05 was considered significant.

Observation 1: Task grade. Agents using direct invocation achieved a higher overall mean grade (0.944) compared with agents using code-execution invocation (0.922), with discovery tasks showing the largest difference (see Fig. 3).

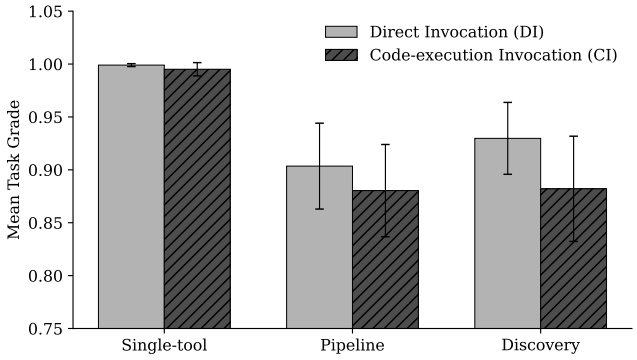

Fig. 3. Mean task grade by invocation modality and task type. Error bars indicate 95% confidence intervals.

Analysis. Although agents using direct invocation consistently outperformed those using code-execution invocation in raw grade, a one-way ANOVA indicates no significant main effect of orchestration modality on task grade ($F = 2.456, p = 0.118$). This suggests that code-execution invocation preserves output quality while potentially offering the cost advantages examined below.

Observation 2: Normalized efficiency. Agents using code-execution invocation are significantly more efficient on pipeline and discovery tasks, but less efficient on single-tool tasks (see Fig. 4 and Table 2).

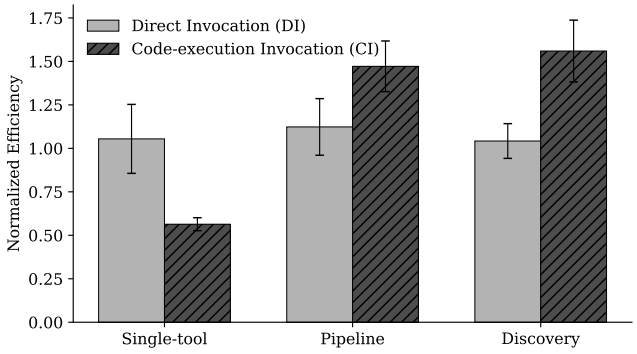

Fig. 4. Normalized efficiency by invocation modality and task type. Error bars indicate 95% confidence intervals. Efficiency was normalized such that single-agent DI equals 1.0 within each task type

Table 2. Normalized efficiency by invocation modality and task type.

Task type	**DI**	**CI**	t	p	d
Single-tool	1.055 ± 0.198	0.564 ± 0.037	-4.823	<0.001	-0.675
Pipeline	1.123 ± 0.163	1.472 ± 0.146	3.162	0.002	0.430
Discovery	1.042 ± 0.100	1.560 ± 0.178	5.059	<0.001	0.803

Analysis. Agents using code-execution invocation were more efficient on both pipeline and discovery tasks due to their ability to batch multiple operations within a single program, thereby reducing the cumulative token-usage overhead associated with sequential direct MCP calls. This yields up to a 1.5× improvement in quality-per-token. The effect is particularly pronounced for discovery tasks ($d = 0.803$, large), as the programmatic enumeration of tools in code-execution invocation avoids repeatedly loading all tool schemas into the agent's context.

Observation 3 - Token usage. Agents using direct invocation consumed an average of 31,161 tokens, compared with 21,576 tokens for agents using code-execution invocation (a 1.44× reduction). These savings are statistically significant for pipeline tasks, but not for single-tool or discovery tasks (see Fig. 5 and Table 3).

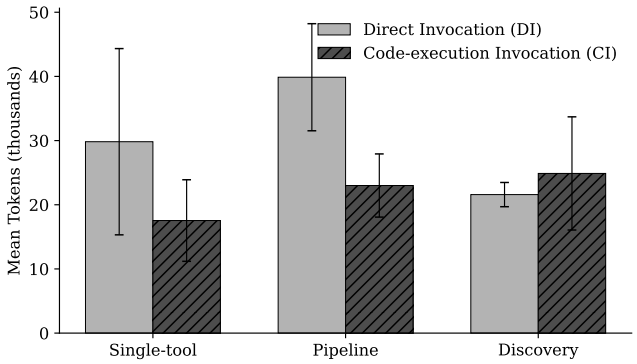

Fig. 5. Mean token usage (in thousands) by invocation modality and task type. Error bars indicate 95% confidence intervals.

Table 3. Mean token usage (in thousands) by invocation modality and task type.

Task type	**DI**	**CI**	t	p	d
Single-tool	29.8 ± 14.5	17.5 ± 6.4	1.538	0.126	0.215
Pipeline	39.9 ± 8.3	23.0 ± 4.9	3.453	<0.001	0.470
Discovery	21.6 ± 1.9	24.9 ± 8.8	−0.728	0.469	−0.116

Analysis. Agents using code-execution invocation achieved the largest token savings on pipeline tasks (Fig. 5), supporting the theoretical argument presented in [5]: batching sequential tool calls within a single code-execution step avoids the token overhead of routing each intermediate result through the LLM context. However, for discovery tasks, agents using code-execution invocation consumed a similar number of tokens as those using direct invocation, likely due to the additional overhead of a single agent managing a large toolset. A one-way ANOVA ($F = 7.205$, $p = 0.008$) confirms that invocation modality has a significant effect on token consumption when considered in aggregate.

5 Conclusions

This work provides one of the earliest reproducible empirical comparisons of two orchestration modalities in agentic AI systems: direct invocation and code-execution invocation. In addition, it introduces an evaluation framework for assessing orchestration strategies in terms of token usage and task quality, using a heterogeneous task suite and an LLM-as-a-judge grader across both single- and multi-agent architectures.

The empirical results show that code-execution invocation reduces token consumption while preserving output quality. Moreover, it achieves higher efficiency on complex tasks, particularly pipeline tasks, when compared with direct invocation.

As a direction for future work, experiments with different state-of-the-art AI models are planned to explore potential model-dependent performance differences.

Acknowledgments. [Hidden for double-blind review]. It is acknowledged that grammar was checked and adjusted using Microsoft Copilot.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Acharya, D.B., Kuppan, K., Divya, B.: Agentic ai: Autonomous intelligence for complex goals - a comprehensive survey. IEEE Access **13**, 18912–18936 (2025)
2. Amazon Web Services: Building agentic ai workflows with mcp, agentcore, and bedrock: A hands-on guide (2025), `https://builder.aws.com/content/37INxVLnjoFrqpr0W9vlKdlpBZs/building-agentic-ai-workflows-with-mcp-agentcore-and-bedrock-a-hands-on-guide`, accessed: 2026-03-26
3. Amazon Web Services, Inc.: Amazon bedrock: The platform for building generative ai applications and agents at production scale. `https://aws.amazon.com/bedrock/` (2026), accessed: 2026-03-27
4. An, S., Ma, Z., Lin, Z., Zheng, N., Lou, J.G., Chen, W.: Make your llm fully utilize the context. Advances in Neural Information Processing Systems **37**, 62160–62188 (2024)
5. Anthropic: Code execution with mcp: Building more efficient ai agents. `https://www.anthropic.com/engineering/code-execution-with-mcp` (2025), published November 4, 2025. Accessed: 2026-03-27
6. Anthropic: Introducing advanced tool use on the claude developer platform (2025), `https://www.anthropic.com/engineering/advanced-tool-use`, accessed: 2026-03-29
7. Bandi, A., Kongari, B., Naguru, R., Pasnoor, S., Vilipala, S.V.: The rise of agentic ai: A review of definitions, frameworks, architectures, applications, evaluation metrics, and challenges. Future Internet **17**(9), 404 (2025)

8. Dobriy, D., Bauer, F., Azzam, A., Banerjee, D., Polleres, A.: Agentic sparql: Evaluating sparql-mcp-powered intelligent agents on the federated kgqa benchmark. arXiv preprint arXiv:2603.06582 (2026)
9. Dwivedi, Y.K., Helal, M.Y., Elgendy, I.A., Alahmad, R., Walton, P., Suh, A., Singh, V., Jeon, I.: Agentic ai systems: What it is and isn't. Global Business and Organizational Excellence **45**(3), 253–263 (2026)
10. Godavarthi, S., Nagvekar, R.: Investment banking with genai. In: Transforming Financial Services with Generative AI: From Strategy and Design to Practical Applications, pp. 191–218. Springer (2026)
11. Guo, Z., Xu, B., Zhu, C., Hong, W., Wang, X., Mao, Z.: Mcp-agentbench: Evaluating real-world language agent performance with mcp-mediated tools. In: Proceedings of the AAAI Conference on Artificial Intelligence. vol. 40, pp. 30888–30896 (2026)
12. Hou, X., Zhao, Y., Wang, S., Wang, H.: Model context protocol (mcp): Landscape, security threats, and future research directions. ACM Trans. Softw. Eng. Methodol. (Feb 2026), just accepted.
13. Huang, J., Zhu, H., Xu, C., Zhan, T., Xie, Q., Huang, J.: Auditwen: An open-source large language model for audit. In: Proceedings of the 23rd Chinese National Conference on Computational Linguistics. pp. 1351–1365 (2024)
14. Joel, S., Wu, J., Fard, F.: A survey on llm-based code generation for low-resource and domain-specific programming languages. ACM Trans. Softw. Eng. Methodol. (Oct 2025), just accepted.
15. Kong, Q., Zhang, X., Yang, Z., Gao, N., Liu, C., Tong, P., Cai, C., Zhou, H., Zhang, J., Chen, L., et al.: Mobileworld: Benchmarking autonomous mobile agents in agent-user interactive and mcp-augmented environments. arXiv preprint arXiv:2512.19432 (2025)
16. LF Projects, LLC: What is the model context protocol (mcp)? (2024), `https://modelcontextprotocol.io/docs/getting-started/intro`, accessed: 2026-03-29
17. Mo, G., Zhong, W., Chen, J., Yuan, Q., Chen, X., Lu, Y., Lin, H., He, B., Han, X., Sun, L.: Livemcpbench: Can agents navigate an ocean of mcp tools? arXiv preprint arXiv:2508.01780 (2025)
18. Naveed, H., Khan, A.U., Qiu, S., Saqib, M., Anwar, S., Usman, M., Akhtar, N., Barnes, N., Mian, A.: A comprehensive overview of large language models. ACM Transactions on Intelligent Systems and Technology **16**(5), 1–72 (2025)
19. Raheem, T., Hossain, G.: Agentic ai systems: Opportunities, challenges, and trustworthiness. In: 2025 IEEE International Conference on Electro Information Technology (eIT). pp. 618–624. IEEE (2025)
20. Ratner, N., Levine, Y., Belinkov, Y., Ram, O., Magar, I., Abend, O., Karpas, E., Shashua, A., Leyton-Brown, K., Shoham, Y.: Parallel context windows for large language models. In: Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 6383–6402 (2023)
21. Sapkota, R., Roumeliotis, K.I., Karkee, M.: Ai agents vs. agentic ai: A conceptual taxonomy, applications and challenges. Information Fusion p. 103599 (2025)
22. Wu, Y., Gu, Y., Feng, X., Zhong, W., Xu, D., Yang, Q., Liu, H., Qin, B.: Extending context window of large language models from a distributional perspective. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing. pp. 7288–7301 (2024)
23. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K.R., Cao, Y.: React: Synergizing reasoning and acting in language models. In: The eleventh international conference on learning representations. pp. 1–33 (2022)